Explicación de React JS

En este documento se explican los conceptos fundamentales de React JS, la biblioteca más utilizada para el desarrollo de interfaces de usuario (frontend). Aunque en mi proyecto Android utilicé Jetpack Compose (el framework moderno de Android), los conceptos que se explican a continuación son los mismos que se aplican en React, ya que ambos siguen el paradigma declarativo.

1. Diferencia entre React y JSX

- React es una biblioteca de JavaScript (desarrollada por Facebook/Meta) que sirve para construir interfaces de usuario interactivas y reutilizables. Se basa en componentes y utiliza un Virtual DOM para actualizar la interfaz de forma eficiente.

- JSX (JavaScript XML) es una extensión de sintaxis que permite escribir código que parece HTML directamente dentro de archivos JavaScript.
  JSX no es HTML, es azúcar sintáctico que se transpila a llamadas de React.createElement().

Ejemplo práctico:

```
// Esto es JSX
const elemento = <h1>Hola mundo desde React</h1>;

// Sin JSX (lo que realmente hace React por detrás)
const elemento = React.createElement('h1', null, 'Hola mundo desde React');
```

Resumen de la diferencia:
React = la biblioteca / el motor.
JSX = la forma cómoda de escribir la interfaz (parecido a HTML).

2. ¿Qué son las clases en React?

- **Componentes de Clase (Class Components):**
  Son clases de JavaScript que extienden de React.Component.
  Tienen estado (this.state), métodos del ciclo de vida (componentDidMount, componentDidUpdate, etc.) y usan this para acceder a props y estado.

- **Componentes Funcionales (Functional Components):**
  Son funciones de JavaScript. Desde la llegada de los Hooks (2019), son la forma recomendada y más usada actualmente.

Ejemplo práctico de Componente de Clase:

```
import React, { Component } from 'react';

class Contador extends Component {
  constructor(props) {
```

```
    super(props);
    this.state = { contador: 0 };
  }

  incrementar = () => {
    this.setState({ contador: this.state.contador + 1 });
  }

  render() {
    return (
      <div>
        <p>Contador: {this.state.contador}</p>
        <button onClick={this.incrementar}>+1</button>
      </div>
    );
  }
}

export default Contador;
```

Conclusión actual: Hoy en día se prefieren los componentes funcionales con Hooks (useState, useEffect, etc.) porque son más simples, legibles y reutilizables.

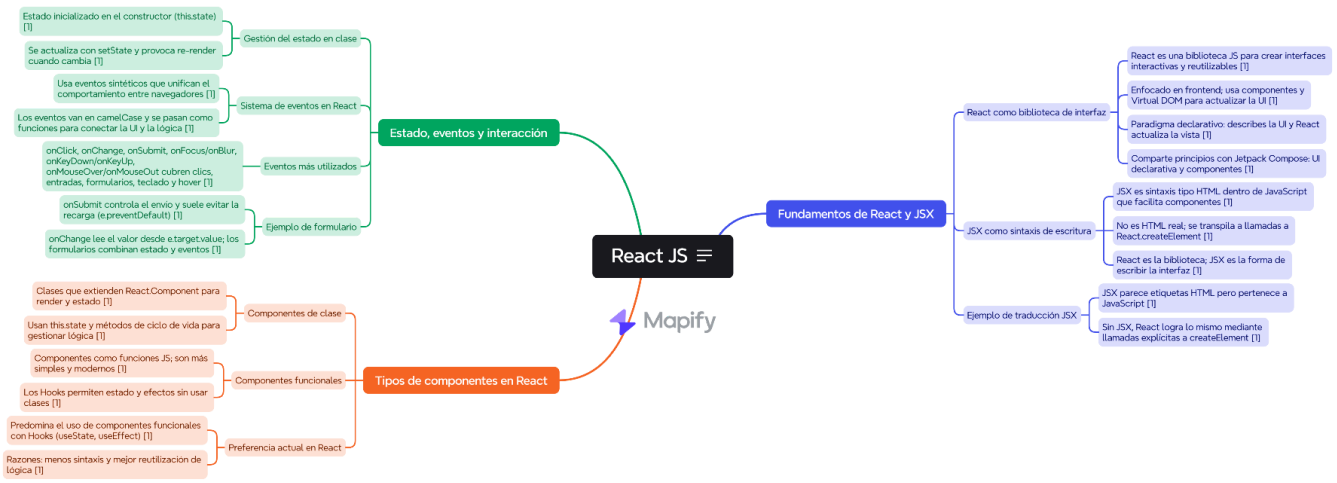
3. Principales eventos en React

React utiliza un sistema de eventos sintéticos (SyntheticEvent) que funciona de forma similar en todos los navegadores.
Los eventos se escriben en camelCase y se pasan como funciones (no como strings).
Eventos más utilizados:

Evento	Descripción	Ejemplo de uso
`onClick`	Click del mouse	Botones
`onChange`	Cambio en input, textarea, select	Formularios
`onSubmit`	Envío de formulario	`<form>`
`onFocus` / `onBlur`	Elemento gana o pierde foco	Validaciones
`onKeyDown` / `onKeyUp`	Teclas presionadas	Atajos de teclado
`onMouseOver` / `onMouseOut`	Paso del mouse	Efectos hover

Ejemplo práctico:

```
function Formulario() {
  const handleSubmit = (e) => {
    e.preventDefault();        // Evita recargar la página
    console.log("Formulario enviado");
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" onChange={(e) => console.log(e.target.value)} />
      <button type="submit">Enviar</button>
    </form>
  );
}
```

Mapa Conceptual de React JS